
H09: 結果の取得と出力 その1

演習パッケージ: P09_Scoring1

Geant4 11.3.0準拠

Geant4講習会資料: HEP/NP/Space 2025

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習トピックス

- 水ファントムに200MeVの陽子を入射しBragg ピークを測定する。このテーマを3つのプログラムで解決する。
 1. コマンドベーススコアリング:
 - ✓ 自分でC++プログラミングをせずに、よく使われるスコアを取得できるコマンドベーススコアリングを実行し、水ファントム中での陽子の全吸収エネルギーを得て、Bragg ピークを観察する
 2. 同じ課題を有感検出器を使って解決する第一歩
 - ✓ ジオメトリの作成
 - ✓ SensitiveDetectorの実装まで
 3. Bragg ピークを表示するためにデータ保管庫を使う
 - ✓ 上の実装を出発点にして
 - ✓ スコアの蓄積と出力
 - ✓ 出力ファイルの解析

演習目次

1. P09_Scoring1/WaterBox_CBS

- Command-Based Scoring を試してみる
- CSVファイルをプログラム無しで可視化する

2. P09_Scoring1/Bragg_SD

- ✓ Geometry SensitiveDetector の実装
- ✓ 結果は端末出力だけ

3. P09_Scoring1/Bragg_DataStore

- 独立するデータストアを使うために新しいクラスの作成
- それを使うために他のクラスの改造
- CSVファイル出力
- 可視化

演習プログラムP09Scoring1のサブディレクトリ構造の確認

- **課題**: 1 P09Scoring1の下に次のような2つのプログラムがあることを確認する。
 1. WaterBox_CBS
 - P05_FirstStep/WaterBox を下敷きにしてCommand-Based Scoringを実行する
 2. Bragg_SD
 - 上と同じ結果を得るためにWaterBoxをG4PVReplicaを使って同等のvoxelを作る
 - 300個のボクセルでのエネルギーデポジットを求めるSensitiveDetectorを作成し、結果を端末に出力する
 3. Bragg_DataStore
 - 上と同じ結果を求めるために
 - データ保管庫を用意し、
 - SensitiveDetectorはそこにスコアを上げ、
 - DataStoreに集積されたデータをランの終わりにCSVファイルに書き出す
 - CSVファイルをもとにBragg peakを描画する

P09_Scoring1/WaterBox_CBS

Command Based Scoring を使ってみる



1. [P09_Scoring1/WaterBox_CBS](#)
 - Command-Based Scoring を使ってみる
 - CSVファイルをプログラム無しで可視化する
2. [P09_Scoring1/Bragg_SD](#)
 - ✓ Geometry [SensitiveDetector](#) の実装
 - ✓ 結果は端末出力だけ
3. [P09_Scoring1/Bragg_DataStore](#)
 - 独立するデータストアを使うために新しいクラスの作成
 - それを使うために他のクラスの改造
 - CSVファイル出力
 - 可視化

プログラムの構成

課題: プログラム構成を理解する

- ✓ これは、演習 (P05_FirstStep/WaterBox) の全コードが P09_Scoring1/WaterBox_CBS にコピーし、水ファントムの寸法を 30cm 立方とした。またメインプログラムだけ一部変更して G4ScoringManager を使う設定とする。。 TestBench には陽子の Bragg peak を観察するためのマクロファイルを置いてある

```
$ cd ~/Gean4Tutorial_2025/P09_Scoring1/WaterBox_CBS
$ ls
source/  util/  TestBench/
```

- 各ディレクトリ下のすべてのファイルを確認する

```
$ls source
  Application_Main.cc CMakeLists.txt include src
$ls source/src
  Geometry.cc PrimaryGenerator.cc UserActionInitialization.cc
$ls source/include
  Geometry.hh PrimaryGenerator.hh UserActionInitialization.hh
$ls TestBench
  GlobalSetup.mac primaryGeneratorSetup.mac proton200MeV.mac
  verboseSetup.mac visSetup_Simplex.mac
```

メインプログラムを一部変更

マクロファイル

演習プログラムをコンパイルしてTestBench下で立ち上げる

課題: P09_Scoring1/WaterBox_CBSのソースコードをビルドして実行する

1) P09_Scoring1/WaterBox_CBSのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P09_Scoring1/WaterBox_CBS
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

- ビルド手順は全演習を通して共通のビルド自動実行のscript **util/Build.sh** を使う
- **util/Build.sh**を実行して以下のようにソースをビルドする

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```


マクロファイル proton200MeV.mac を実行

課題: 200MeV 陽子の吸収線量をスコアするマクロファイルを実行

1) Qt画面のコマンド窓に以下のコマンドを入力

```
/control/execute proton200MeV.mac
```

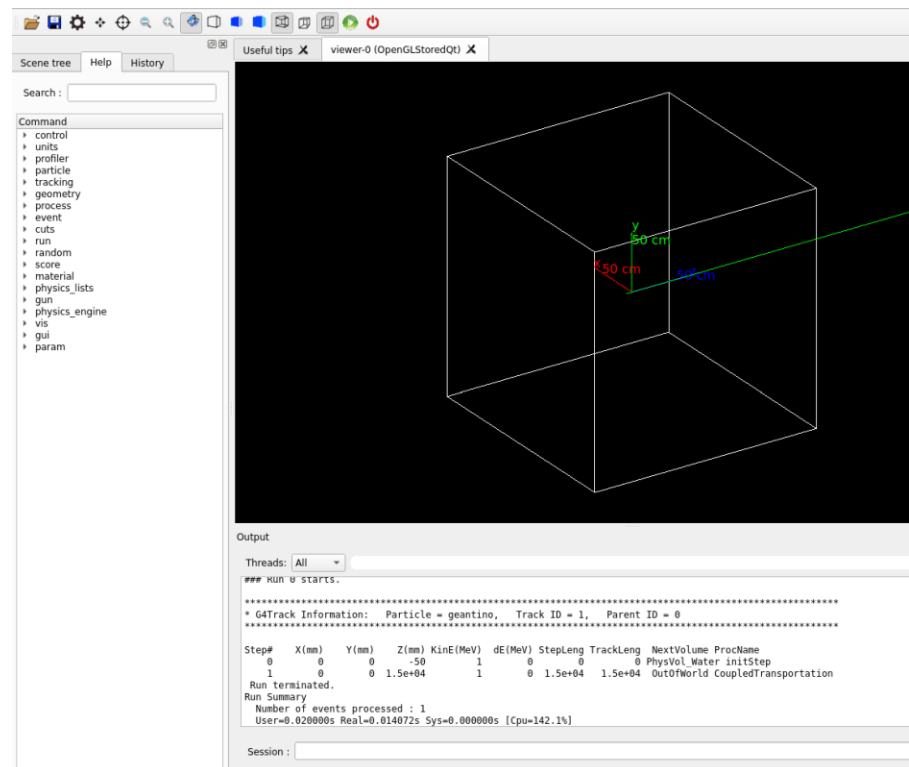
- GlobalSetup.mac はメインプログラムが立ち上がった時に実行済

2) アプリを終了

```
exit
```

3) TestBench/ の下に “proton-dDep.csv” ファイルを確認

```
$ ls
... proton-dDep.csv...
$ wc proton-dDep.csv
```



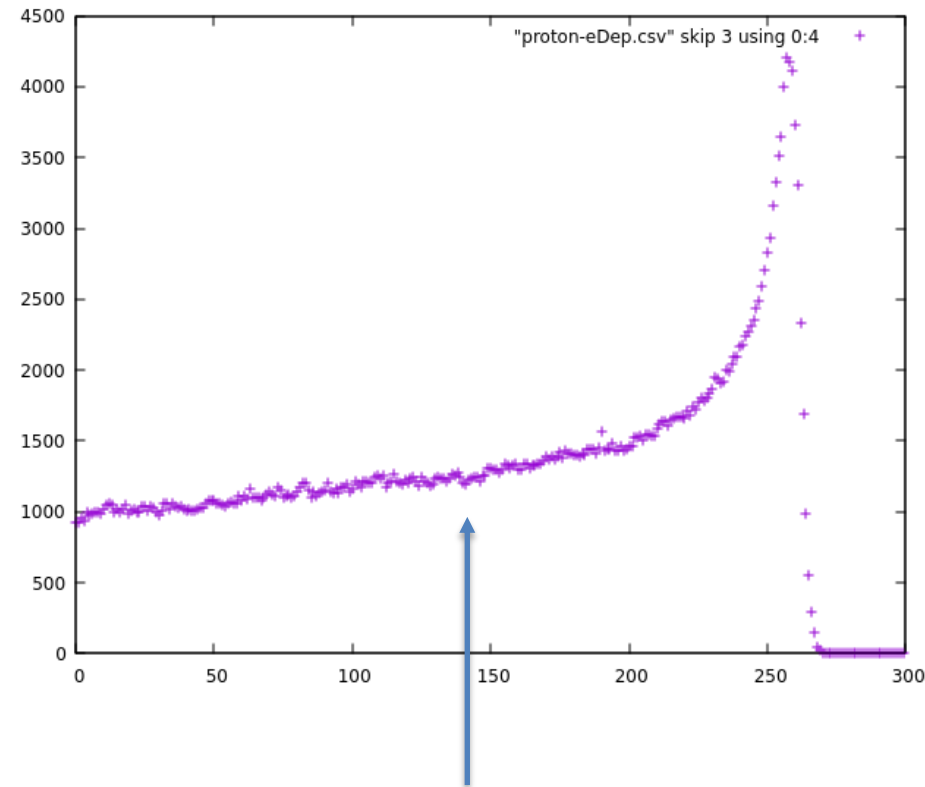
Proton-dDep.csvファイルからヒストグラムを作成する

出力 proton-dDep.csv の冒頭部分

```
# mesh name: boxMesh_1
# primitive scorer name: eDep
# iX, iY, iZ, total(value) [MeV], total(val^2), entry
0,0,0,920.8011679802574,773.4926002404895,2000
0,0,1,926.4099572510512,588.4595861134717,2000
0,0,2,960.8945679974946,1816.424493107643,2000
0,0,3,934.0524951082191,702.3284235342687,2000
0,0,4,1000.454077760711,1715.366284158613,2000
0,0,5,972.4285785072846,1113.167024615977,1999
0,0,6,986.6208489385363,1409.500151809499,1999
0,0,7,997.4981749162156,1418.96460972925,1998
0,0,8,999.4805490161147,2084.723872072736,1997
0,0,9,983.2125944069894,1300.498781858488,1996
0,0,10,1020.330696431658,2876.570763173628,1996
0,0,11,1045.818153781285,2149.054942287487,1996
0,0,12,1055.737368218362,2571.330241115404,1996
```

最初の3行のコメント行の後、第4カラムをプロットする

課題 gnuplotでの描画



```
gnuplot> set datafile separator ","
gnuplot> plot "proton-eDep.csv" skip 3 using 0:4
```

Application_Main.ccへの変更の主要部分

課題: Command Base Scoringに必要なメインプログラムへの追加コード赤枠部分を読む

Command Based Scoring を使うためのC++コードはこれがすべての変更である。

```
//+++++
// Geant4 Application: Tutorial course for Hep/Space/Medicine Users
//+++++
#include "Geometry.hh"
#include "UserActionInitialization.hh"
#include "G4ScoringManager.hh"
#include "G4RunManager.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "FTFP_BERT.hh"

//-----
int main( int argc, char** argv )
//-----
{
    // Construct the default run manager
    auto runManager = new G4RunManager();
    // Activate UI-command base scorer
    G4ScoringManager * scManager = G4ScoringManager::GetScoringManager();
    scManager->SetVerboseLevel(1);
}
```

G4RunManager のinstantiationの直後に
G4ScoringManagerへのポインターを得ること

proton200MeV.mac スコアリング・マクロの内容

課題: Braggピーク観測用マクロファイル

```
$ cd ../TestBench  
$ less proton200MeV.mac
```

```
# define scoring mesh "boxMesh_1"  
#  
/score/create/boxMesh boxMesh_1  
# its size half lengths and number of bins in x, y and z  
/score/mesh/boxSize 15. 15. 15. cm  
/score/mesh/nBin 1 1 300  
#scores given name "eDep" and "dDep"  
/score/quantity/energyDeposit eDep  
/score/quantity/doseDeposit dDep  
/score/close  
/score/list  
# verbosity  
/control/verbose 2  
/run/verbose 1  
/tracking/verbose 0  
## particle  
/gun/particle proton  
/gun/position 0. 0. -15. cm  
/gun/direction 0. 0. 1.  
/gun/energy 200 MeV  
/run/beamOn 2000  
# Dump scorers to a file  
/score/dumpQuantityToFile boxMesh_1 dDep proton-dDep.csv  
/score/dumpQuantityToFile boxMesh_1 eDep proton-eDep.csv
```

スコア用メッシュ切り

“boxMesh_1” という名前の立方体で
寸法は15 x 15 x 15 cm (実寸の半分)
Bin数はx, y, z 方向に1, 1, 300とするの
で1ビンのz方向長さは 1mm となる

エネルギー付与(eDep)と線量付与(dDep)

Verbosity, 特にtrackingのお喋りを抑制する

WaterBoxの左端から右方向へ200MeVの
陽子を入射する。陽子の飛程を勘案してこ
のエネルギーを選んである。変えてみるの
もよいだろう。

ランの後eDep, dDepをCSV形式で出力する

Command Based Scoring まとめ

1. 提供されているスコアが自分の欲しいものであって、イベント単位ではなく**ラン**ごとの積算スコアでよい場合には使える
 - 詳しくはユーザマニュアル。フィルタ付きのスコアも使える。
2. 実物質空間にかかわりなく自由にメッシュを切って置ける。
 - 今の例では水ファントムと同じ寸法の一次元メッシュを切ってみた。Voxel メッシュに変更するは簡単。
 - 動径方向のスコアが欲しい場合には、円筒座標メッシュを切ればよい。
 - 水ファントムの外への漏洩を求めるには、そこに大きなメッシュを切ればよい
3. C++ コードで書くのは**Geometryだけでよい**。Geometryは自分だけのものだから書くしかない。スコアリングのためにはmain.ccで二行加えるだけ。
4. スコアをCSVファイルで出力できるのが重要で、外部の解析アプリに渡すといろんなデータ解析が可能になる。

P09_Scoring1/Bragg_SD

1. [P09_Scoring1/WaterBox_CBS](#)

- Command-Based Scoring を試してみる
- CSVファイルをプログラム無しで可視化する



2. [P09_Scoring1/Bragg_SD](#)

- ✓ Geometry [SensitiveDetector](#) の実装
- ✓ 結果は端末出力だけ

3. [P09_Scoring1/Bragg_DataStore](#)

- 独立するデータストアを使うために新しいクラスの作成
- それを使うために他のクラスの改造
- CSVファイル出力
- 可視化

P09_Scoring1/Bragg_SDプログラムの概要

■ 演習プログラムの目的

- 有感検出器を使ってP09_Scoring1/WaterBox_CBSと同じ結果を得たいその第一歩として SensitiveDetectorだけを新規作成する

■ P09_Scoring1/Bragg_SD プログラムの構成

- クラス **Geometry** ではWaterBoxをZ方向に300枚に**G4PVReplica**を使って薄切りにし、薄切りの論理ボリウムをSensitiveDetectorとして登録する。これは前のproton200MeV.macのメッシュ切りに合わせるためである。
- クラス **SensitiveDetector**を新規に作成する
- ProcessHits() では (copyNo)番目の薄切り板での全エネルギーデポジット(edep)を求め、**energyMap**に格納する
- 1イベントを走らせ様子を見る。



プログラムの構成

課題: プログラム構成を確認する

- ✓ これは、演習P09_Scoring1/WaterBox_CBSとおなじサイズの水ファントムをZ方向に300枚の薄板にReplicateした。その厚さ1mmの薄板をSensitiveDetectorにする。

```
$ cd ~/Gean4Tutorial_2025/P09_Scoring1/Bragg_SD
$ ls
source/  util/  TestBench/
```

- source ディレクトリ下のすべてのファイルを確認する

```
source/:
Application_Main.cc  CMakeLists.txt  include  src

source/include:
Geometry.hh  PrimaryGenerator.hh  SensitiveDetector.hh  UserActionInitialization.hh

source/src:
Geometry.cc  PrimaryGenerator.cc  SensitiveDetector.cc  UserActionInitialization.cc
```

Geometryを一部変更,
SensitiveDetectorを追加

演習プログラムをコンパイルしてTestBench下で立ち上げる

課題: P09_Scoring1/Bragg_SDのソースコードをビルドして実行する

1) P09_Scoring1/WaterBox_CBSのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P09_Scoring1/Bragg_SD
```

2) ソースをビルド

- ビルド手順は全演習を通して共通のビルド自動実行のscript **util/Build.sh** を使う
- **util/Build.sh**を実行して以下のようにソースをビルドする

```
$ ./util/Build.sh  
$ ls  
bin/  build/  source/  util/  TestBench/
```

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench  
$ ../bin/Application_Main
```

マクロファイル p200.mac を実行

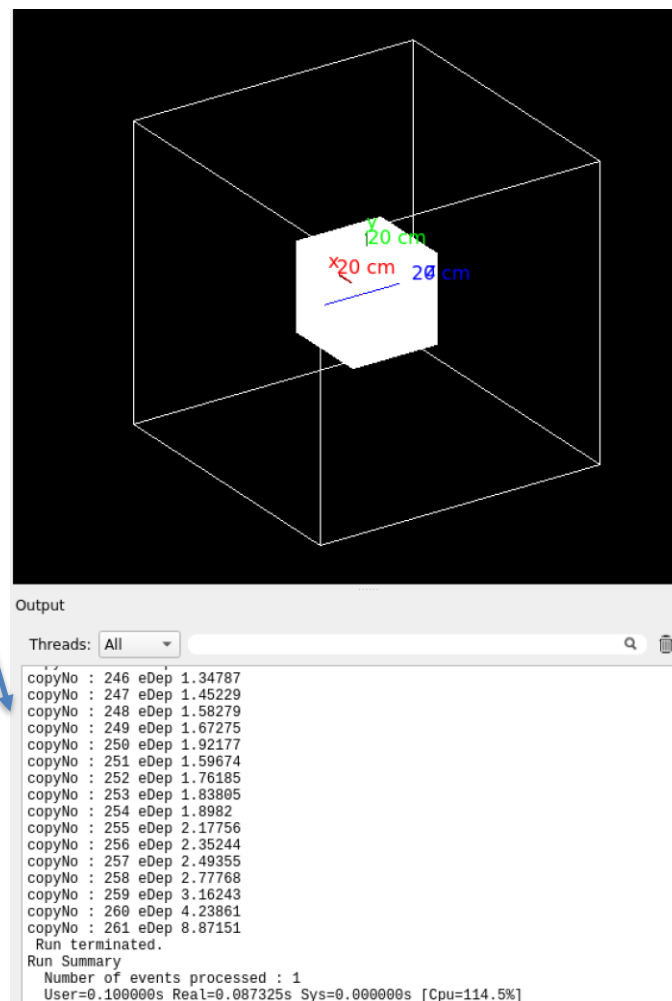
課題: 200MeV 陽子の吸収線量をスコアするマクロファイルを実行

1) Qt画面のコマンド窓に以下のコマンドを入力

```
/control/execute p200.mac
```

- 1イベントだけランする。
- 右図のようにcopyNoとeDepの対が出力されるだけ。
- SensitiveDetectorでは多数のイベントを走らせて蓄積するようには作っていない。それは次のBragg_DataStoreでやる
- ここではSensitiveDetector で右のような出力をするためにどうやったかを見るだけとする。

2) アプリを終了



Output

Threads: All

copyNo	:	246	eDep	1.34787
copyNo	:	247	eDep	1.45229
copyNo	:	248	eDep	1.58279
copyNo	:	249	eDep	1.67275
copyNo	:	250	eDep	1.92177
copyNo	:	251	eDep	1.59674
copyNo	:	252	eDep	1.76185
copyNo	:	253	eDep	1.83805
copyNo	:	254	eDep	1.8982
copyNo	:	255	eDep	2.17756
copyNo	:	256	eDep	2.35244
copyNo	:	257	eDep	2.49355
copyNo	:	258	eDep	2.77768
copyNo	:	259	eDep	3.16243
copyNo	:	260	eDep	4.23861
copyNo	:	261	eDep	8.87151

Run terminated.

Run Summary

Number of events processed : 1

User=0.100000s Real=0.087325s Sys=0.000000s [Cpu=114.5%]

Geometry.cc

```
#include "Geometry.hh"
#include "G4Box.hh"
#include "G4LogicalVolume.hh"
#include "G4PVPlacement.hh"
#include "G4PVReplica.hh"
#include "G4Material.hh"
#include "G4NistManager.hh"
#include "G4SystemOfUnits.hh"
#include "G4SDManager.hh"
#include "SensitiveDetector.hh"
```

赤色は追加したクラス定義

```
Geometry::Geometry() = default;
Geometry::~~Geometry() = default;
```

```
G4VPhysicalVolume* Geometry::Construct() {
    G4NistManager* nist = G4NistManager::Instance();
```

```
    // World
```

```
    G4Material* air = nist->FindOrBuildMaterial("G4_AIR");
```

略

```
    // Water Phantom
```

```
    G4Material* water = nist->FindOrBuildMaterial("G4_WATER");
```

```
    G4Box* phantomBox = new G4Box("Phantom", 15 * cm, 15 * cm, 15 * cm);
```

```
    G4LogicalVolume* phantomLogic = new G4LogicalVolume(phantomBox, water, "Phantom");
```

```
    new G4PVPlacement(nullptr, {}, phantomLogic, "Phantom", worldLogic, false, 0);
```

課題: Geometry: SensitiveDetector アサインを読む

// Slices

```
G4Box* sliceBox = new G4Box("Slice", 15 * cm, 15 * cm, 0.5 * mm);  
G4LogicalVolume* sliceLogic = new G4LogicalVolume(sliceBox, water, "Slice");  
new G4PVReplica("Slices", sliceLogic, phantomLogic, kZAxis, 300, 1 * mm);
```

薄切り

レプリケーション

sliceLogic を SensitiveDetector にする

// Assign sensitive detector

```
G4SDManager* sdManager = G4SDManager::GetSDMpointer();  
SensitiveDetector* sd = new SensitiveDetector("WaterPhantom");  
sdManager->AddNewDetector(sd);  
sliceLogic->SetSensitiveDetector(sd);  
return worldPhys;
```

薄切りの論理ボリュームを有感検出器とする

SensitiveDetector.hh

```
#ifndef SENSITIVE_DETECTOR_HH
#define SENSITIVE_DETECTOR_HH

#include "G4VSensitiveDetector.hh"
#include <map>

class SensitiveDetector : public G4VSensitiveDetector {
public:
    SensitiveDetector(const G4String& name);
    ~SensitiveDetector() override;
    void Initialize(G4HCofThisEvent* hce) override;
    G4bool ProcessHits(G4Step* step, G4TouchableHistory* history) override;
    void EndOfEvent(G4HCofThisEvent* hce) override;

private:
    std::map<G4int, G4double> energyMap;
};

#endif
```

std::mapは、C++の標準テンプレートライブラリ (STL) で提供される連想コンテナで、キーと値のペアを格納し、キーに基づいて値を高速に検索できる。
要素の挿入、更新、削除、検索、サイズの取得、すべての要素の削除などの操作がある

SensitiveDetector.cc の3つの関数を読む

```
#include "SensitiveDetector.hh"
#include "G4Step.hh"
#include "G4TouchableHistory.hh"
SensitiveDetector::SensitiveDetector(const G4String& name)
    : G4VSensitiveDetector(name) {}
SensitiveDetector::~SensitiveDetector() = default;

G4bool SensitiveDetector::ProcessHits(G4Step* step, G4TouchableHistory*) {
    G4double edep = step->GetTotalEnergyDeposit();
    if (edep > 0) {
        G4int copyNo = step->GetPreStepPoint()->GetTouchable()->GetCopyNumber();
        energyMap[copyNo] += edep;
    }
    return true;
}

void SensitiveDetector::Initialize(G4HCofThisEvent*) {
    // at every event the map is cleared
    energyMap.clear();
}

void SensitiveDetector::EndOfEvent(G4HCofThisEvent*) {
    // only to see energyMap
    for (const auto& pair: energyMap){
        G4cout << "copyNo : " << pair.first << " eDep " << pair.second << G4endl;
    }
}
```

copyNoとedep積算値のペアのmap

マップはイベントごとに初期化する

マップの端末出力

P09_Scoring1/Bragg_DataStore

1. [P09_Scoring1/WaterBox_CBS](#)

- Command-Based Scoring を試してみる
- CSVファイルをプログラム無しで可視化する

2. [P09_Scoring1/Bragg_SD](#)

- ✓ Geometry [SensitiveDetector](#) の実装
- ✓ 結果は端末出力だけ



3. [P09_Scoring1/Bragg_DataStore](#)

- 独立するデータストアを使うために新しいクラスの作成
- それを使うために他のクラスの改造
- CSVファイル出力
- 可視化

P09_Scoring1/Bragg_DataStoreプログラムの概要

■ 演習プログラムの目的

- 有感検出器を使ってP09_Scoring1/WaterBox_CBSと同じ結果を得たい。そのためにスコアをどう蓄積し書き出すかに取り組む

■ P09_Scoring1/Bragg_DataStore プログラムの構成

- Bragg_SDを出発点として、energyMapをランを通して積算してランの終わりにCSVファイルに出力する
- そのために、他のクラスとは独立したDataStoreデータ保管庫クラスを用意する
 - energyMap はBragg_SD と同様にstd::map を使う
 - SensitiveDetector::ProcessHit() でenergyMapを積算するメソッドFillMap()
 - UserRunAction::BeginOfRunAction でInitializeMap()
 - UserRunAction::EndOfRunAction でGetMap()
- DataStoreを使うために他のクラスに変更を加える
 - データ保管庫の初期化と読み出しはUserRunActionで行う
 - Application_MainでDataStoreをインスタンス化して
 - UserRunActionの初期化のときにその参照を得る
 - SensitiveDetectorでもその参照を使う
 - Geometry でも同じ
 - UserActionInitializationでも

独立したDataStoreの仕様

- **DataStore** class はデータとそれへのアクセス関数を用意する
 - it has a private store for energy deposit at given copy number plate.
We use private myMap:
`std::map<G4int copyNo, G4double energy>myMap`
 - The class must have public methods ;
`InitializeMap(myMap)` to clear the map at the `RunAction::BeginOfRun`,
 - and public: `GetMap()` which is used at the `RunAction::EndOfRun`.
 - `FillMap()` method which is used by the SensitiveDetector to fill myMap

以下、各クラスでDataStore オブジェクトをどう使うかを実際のコードから調べる。

プログラムの構成 SensitiveDetectorの最小限実装例

課題: プログラム構成を理解する

- ✓ 演習 (P09/Bragg_SD) のコードをもとに、独立したDataStoreを実装する。SensitiveDetector を完成し、Geometryに必要な変更を加えた。

```
$ cd ~/Gean4Tutorial_2025/P09_Scoring1/Bragg_DataStore
$ ls
source/  util/  TestBench/
```

- 各ディレクトリ下のすべてのファイルを確認する
lsコマンドの再帰リスト表示オプションRをつかう

- %ls -R source/

```
source/:
Application_Main.cc  CMakeLists.txt  include  src

source/include:
DataStore.hh  PrimaryGenerator.hh  SensitiveDetector.hh
Geometry.hh  RunAction.hh        UserActionInitialization.hh

source/src:
DataStore.cc  PrimaryGenerator.cc  SensitiveDetector.cc
Geometry.cc  RunAction.cc        UserActionInitialization.cc
```

DataStore.hh/cc は新規追加
他のものは加筆・修正

演習プログラムをコンパイルしてTestBench下で立ち上げる

課題: P09_Scoring1/Bragg_SDのソースコードをビルドして実行する

1) P09_Scoring1/Bragg_SDのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P09_Scoring1/Bragg_DataStore
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

- ビルド手順は全演習を通して共通のビルド自動実行のscript **util/Build.sh** を使う
- **util/Build.sh**を実行して以下のようにソースをビルドする

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

マクロファイル proton200MeV.mac を実行

課題: 200MeV 陽子の吸収線量をスコアするマクロファイルを実行

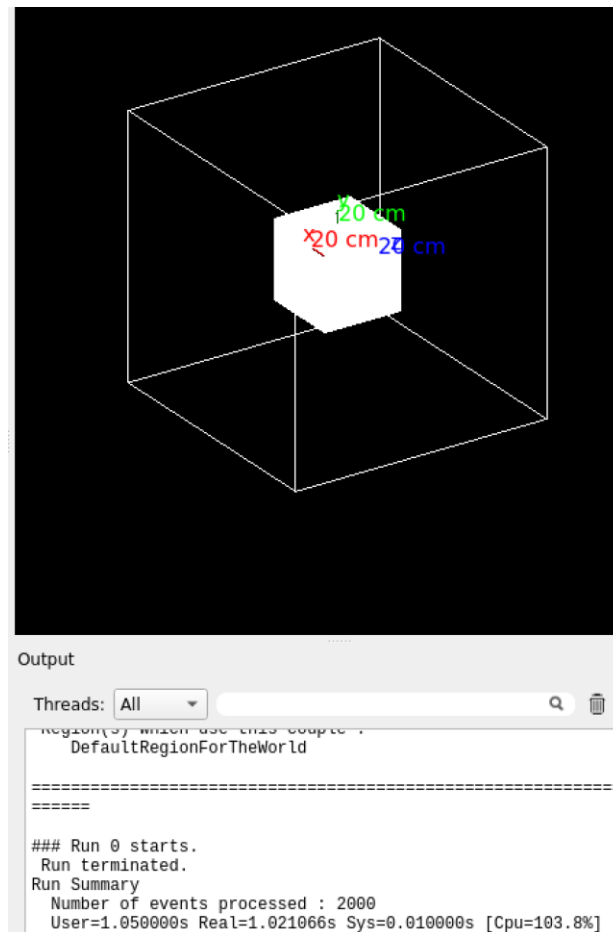
1) Qt画面のコマンド窓に以下のコマンドを入力

```
/control/execute proton200MeV.mac
```

2) 2000イベントでランが終わるので、アプリを終了しBragg.csvを見る

```
%more Bragg.csv
```

```
copyNo, energy
0, 945.675
1, 921.782
2, 928.788
3, 986.152
4, 950.688
5, 925.737
6, 950.653
7, 973.328
8, 984.172
9, 1039.26
10, 989.222
```



課題: CSVファイルをgnuplotで可視化する

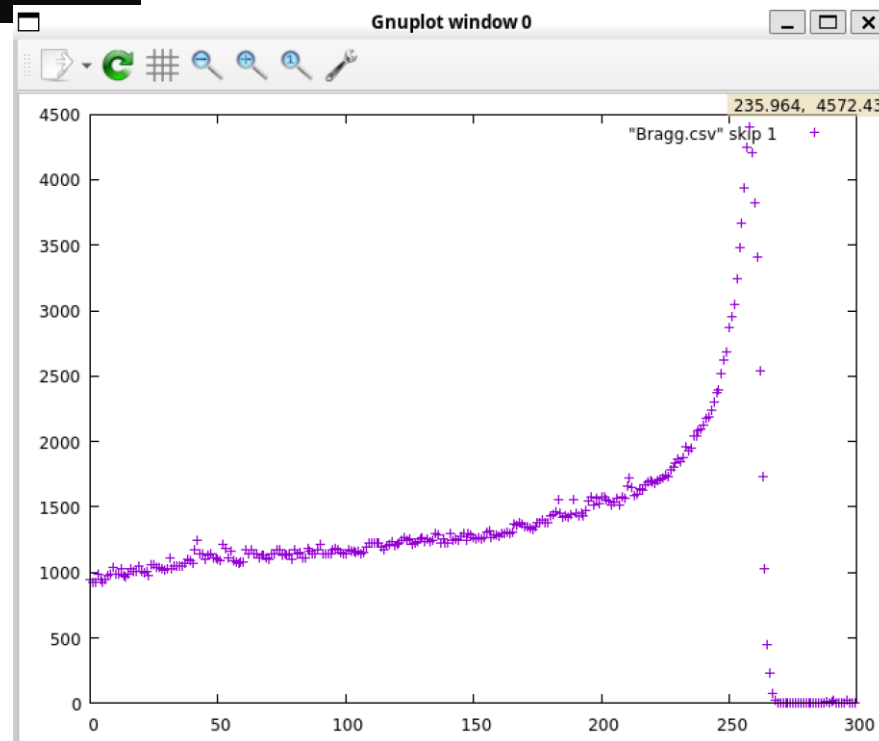
```
GNUPLOT
Version 5.4 patchlevel 2    last modified 2021-06-01

Copyright (C) 1986-1993, 1998, 2004, 2007-2021
Thomas Williams, Colin Kelley and many others

gnuplot home:      http://www.gnuplot.info
faq, bugs, etc:    type "help FAQ"
immediate help:    type "help" (plot window: hit 'h')
```

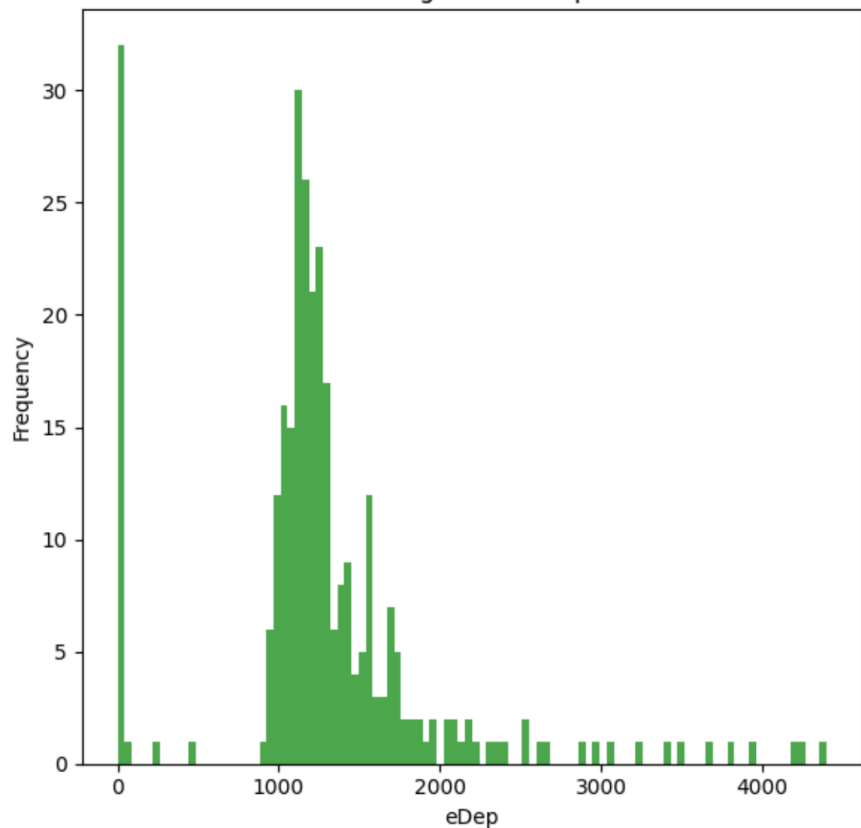
```
Terminal type is now 'qt'
gnuplot> set datafile separator ","
gnuplot> plot "Bragg.csv" skip 1
```

separator “,” はCSV形式を指定、
plot でskip 1は最初のコメント行

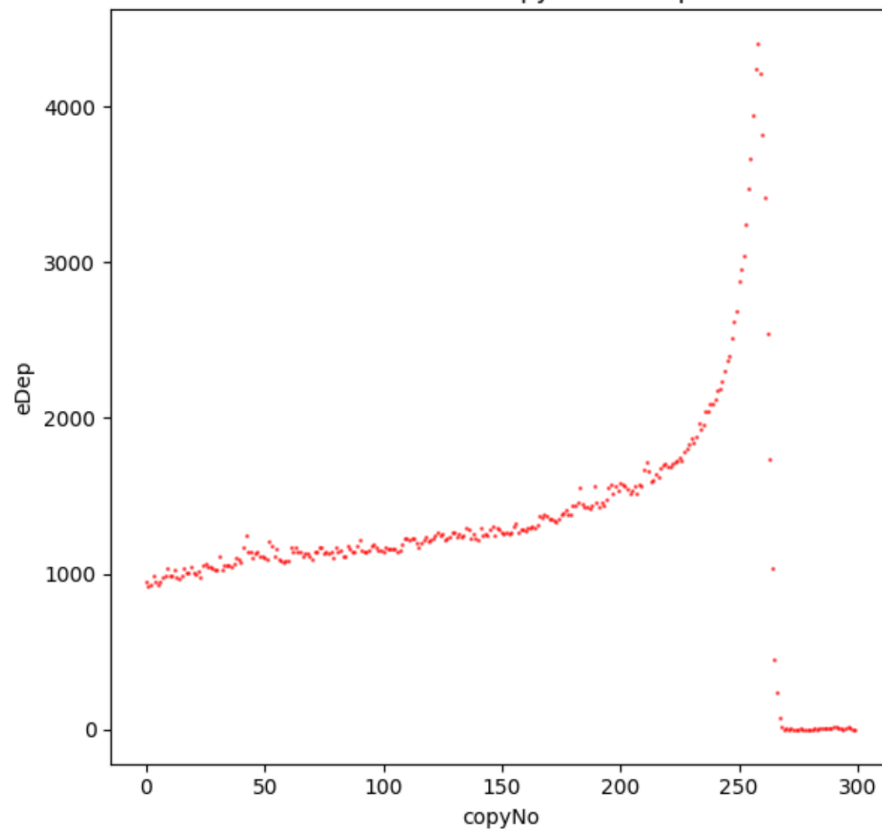


Jupyter lab による解析 Bragg_DataStore.ipynb

Histogram of eDep

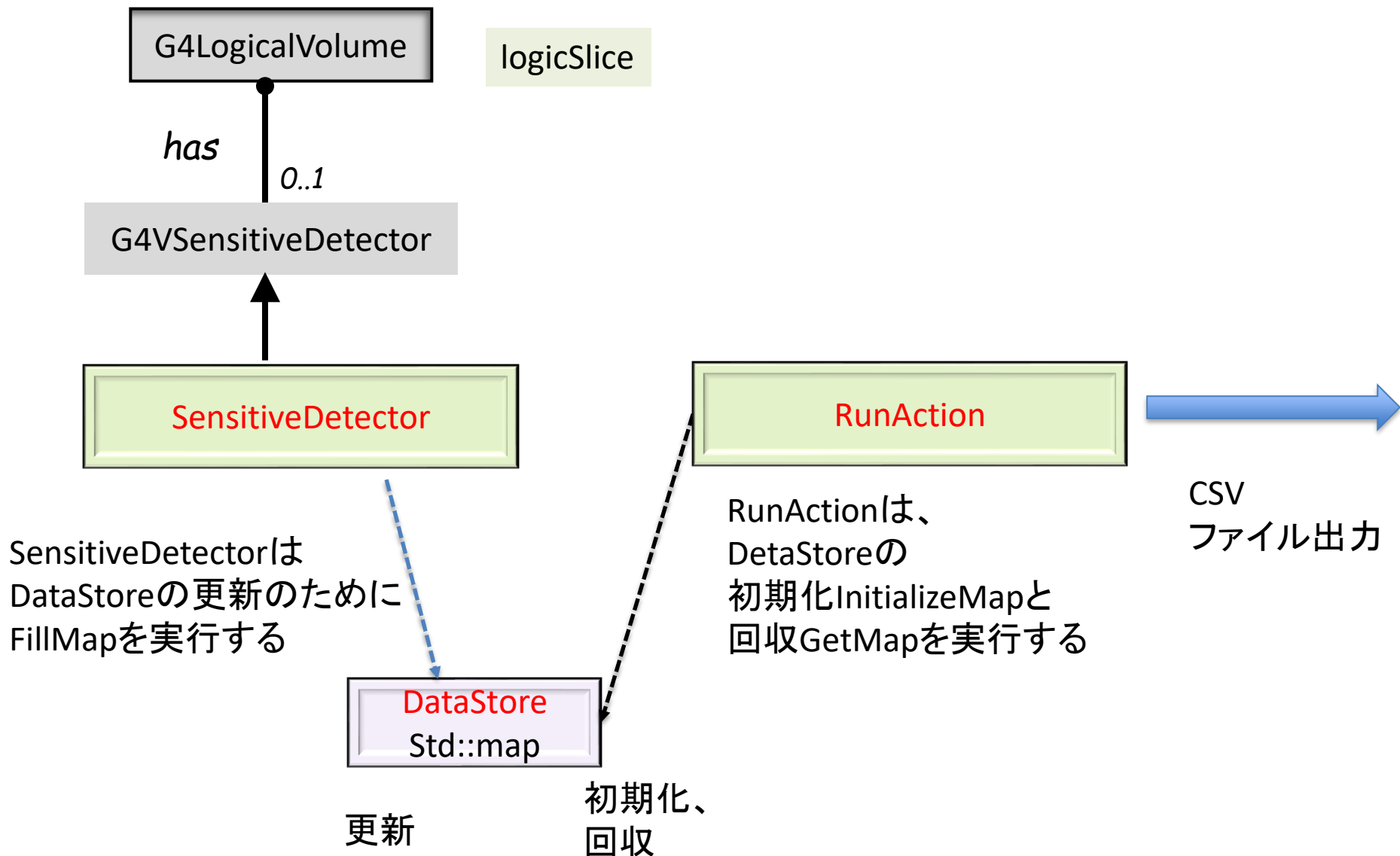


Scatter Plot of copyNo vs eDep



jupyter lab についてはH10で。
このスクリプトはBragg_DataStore/TestBenchにある

独立したデータストアと他のクラスとの関係



課題： 以下のポイントに着目しながらすべてのコードを読む

1. DataStore クラスとSensitiveDetectorおよびRunAction クラスの関係図
2. DataStore クラス
 - `private std::map<G4int, G4double> myMap`
 - `FillMap(), InitializaMap(), GetMap()`
3. SensitiveDetector
 - `SensitiveDetector::SensitiveDetector(const G4String& name, DataStore* ds) : G4VSensitiveDetector(name, dataStore(ds)) {}`
 - `dataStore->FillMap(copyNo, edep);`
4. Geometry
 - `void Geometry::SetDataStore(DataStore* ds) {dataStore = ds;}`
 - `SensitiveDetector* sd = new SensitiveDetector("WaterPhantom", dataStore);`
5. RunAction
 - `RunAction::RunAction(DataStore* ds) : G4UserRunAction(), dataStore(ds) {}`
 - `dataStore->InitializeMap();`
 - `std::map<G4int, G4double> myMap = dataStore->GetMap();`

続き

6. UserActionInitialization

- `::UserActionInitialization(DataStore* ds) : G4VUserActionInitialization(),
dataStore(ds) {}`
- `SetUserAction(new RunAction(dataStore));`

7. Application_Main

- `DataStore* dataStore = new DataStore();`
- `geometry->SetDataStore(dataStore);`
- `runManager->SetUserInitialization(new UserActionInitialization(dataStore));`
- `Delete dataStore;`

```
#ifndef DATASTORE_HH
#define DATASTORE_HH

#include <map>
#include "globals.hh"

class DataStore {
public:
    DataStore();
    ~DataStore();
    void InitializeMap();
    std::map<G4int, G4double> GetMap();
    void FillMap(G4int copyNo, G4double energy);

private:
    std::map<G4int, G4double> myMap;
};

#endif
```

データ保管の核心部

```
#include "DataStore.hh"

DataStore::DataStore() {}
DataStore::~DataStore() {}

void DataStore::InitializeMap() {
    myMap.clear();
}

std::map<G4int, G4double> DataStore::GetMap() {
    return myMap;
}

void DataStore::FillMap(G4int copyNo, G4double energy) {
    myMap[copyNo] += energy;
}
```

RunAction

SensitiveDetector

SensitiveDetector.hh

```
#ifndef SENSITIVEDETECTOR_HH
#define SENSITIVEDETECTOR_HH
```

```
#include "G4VSensitiveDetector.hh"
#include "globals.hh"
#include "G4Step.hh"
#include "DataStore.hh"
```

```
class SensitiveDetector : public G4VSensitiveDetector {
public:
    SensitiveDetector(const G4String& name, DataStore* ds);
    virtual ~SensitiveDetector();
    virtual void Initialize(G4HCofThisEvent* hce) override;
    virtual G4bool ProcessHits(G4Step* step, G4TouchableHistory* history) override;
    virtual void EndOfEvent(G4HCofThisEvent* hce) override;
```

名前とDataStoreオブジェクトの参照

```
private:
    DataStore* dataStore;
};
```

```
#endif
```

SensitiveDetector.cc

```
#include "SensitiveDetector.hh"
#include "G4Step.hh"
#include "G4TouchableHistory.hh"

SensitiveDetector::SensitiveDetector(const G4String& name, DataStore* ds)
: G4VSensitiveDetector(name), dataStore(ds) {}

SensitiveDetector::~SensitiveDetector() {}

void SensitiveDetector::Initialize(G4HCofThisEvent* hce) {}
void SensitiveDetector::EndOfEvent(G4HCofThisEvent* hce) {}

G4bool SensitiveDetector::ProcessHits(G4Step* step, G4TouchableHistory* history) {
    G4int copyNo = step->GetPreStepPoint()->GetTouchable()->GetCopyNumber();
    G4double edep = step->GetTotalEnergyDeposit();
    dataStore->FillMap(copyNo, edep);
    return true;
}
```

SensitiveDetectorのコンストラクタでDataStoreオブジェクトのポインタdsを受け取りそれを**コンストラクタ初期化リスト**を使ってメンバー変数dataStoreに代入する。どのメンバー関数もこれを使えるようになる

Geometry.hh

```
#ifndef GEOMETRY_HH
#define GEOMETRY_HH

#include "G4VUserDetectorConstruction.hh"
#include "globals.hh"

class Geometry : public G4VUserDetectorConstruction {
public:
    Geometry();
    virtual ~Geometry();
    virtual G4VPhysicalVolume* Construct();
    void SetDataStore(DataStore* ds);

private:
    DataStore* dataStore;
};

#endif
```

SetDataStore関数はDataStoreオブジェクトのポインタdsを受け取りそれをメンバー変数dataStoreに代入する仕事

Geometry.cc

```
#include "Geometry.hh"
#include "G4NistManager.hh"
#include "G4Box.hh"
#include "G4LogicalVolume.hh"
#include "G4PVPlacement.hh"
#include "G4PVReplica.hh"
#include "G4SDManager.hh"
#include "G4SystemOfUnits.hh"
#include "SensitiveDetector.hh"
#include "DataStore.hh"

Geometry::Geometry() : G4VUserDetectorConstruction(),
                        dataStore(nullptr) {}

Geometry::~~Geometry() {}

void Geometry::SetDataStore(DataStore* ds) {
    dataStore = ds;
}

G4VPhysicalVolume* Geometry::Construct() {
    // Get materials
    途中略
```

Geometry.cc continued

```
// Slices
G4double slice_thickness = 1.0 * mm;
G4Box* solidSlice = new G4Box("Slice", phantom_size / 2, phantom_size / 2, slice_thickness / 2);
G4LogicalVolume* logicSlice = new G4LogicalVolume(solidSlice, water, "Slice");
new G4PVReplica("PhantomSlice", logicSlice, logicPhantom, kZAxis, 300, slice_thickness);

// Sensitive detector
SensitiveDetector* sd = new SensitiveDetector("WaterPhantom", dataStore);
G4SDManager::GetSDMpointer()->AddNewDetector(sd);
logicSlice->SetSensitiveDetector(sd);

return physWorld;
}
```

名前とDataStoreオブジェクトへの参照をもって
SensitiveDetectorのインスタンスを作る

RunActionの仕様と実装

RunAction class RunAction.hh RunAction.cc

- it must be capable of getting the reference to DataStore
- BeginOfRunAction do Initialize DataStore's myMap.
- EndOfRunAction use DataStore's GetMap method and output the Map to a file in CSV format "copyNo, energy" and output to a CSV file "Bragg.csv"

```
#ifndef RUNACTION_HH
#define RUNACTION_HH

#include "G4UserRunAction.hh"
#include "DataStore.hh"

class RunAction : public G4UserRunAction {
public:
    RunAction(DataStore* ds);
    virtual ~RunAction();
    virtual void BeginOfRunAction(const G4Run*) override;
    virtual void EndOfRunAction(const G4Run*) override;

private:
    DataStore* dataStore;
};

#endif
```

RunAction.hh/cc 読んでみよう。

```
#include "RunAction.hh"
#include "G4Run.hh"
#include <fstream>
```

```
RunAction::RunAction(DataStore* ds) : G4UserRunAction()
    , dataStore(ds) {}
```

```
RunAction::~RunAction() {}
```

```
void RunAction::BeginOfRunAction(const G4Run*) {
    dataStore->InitializeMap();
}
```

RunAction のコンストラクタでDataStoreオブジェクトを
ポインタで受け取り
それをdataStore メンバー変数に保存して使う。

```
void RunAction::EndOfRunAction(const G4Run*) {
    std::map<G4int, G4double> myMap = dataStore->GetMap();
    std::ofstream outFile("Bragg.csv");
    outFile << "copyNo, energy¥n";
    for (const auto& entry : myMap) {
        outFile << entry.first << ", " << entry.second << "¥n";
    }
    outFile.close();
}
```

myMapのすべてのentryにループを回して
CSV形式で出力する

UserActionInitialization.cc

UserActionInitialization class must be properly initialized.
it must be capable of getting the reference to DataStore

```
#include "UserActionInitialization.hh"  
#include "RunAction.hh"  
#include "PrimaryGenerator.hh"
```

```
UserActionInitialization::UserActionInitialization(DataStore* ds) : G4VUserActionInitialization(), dataStore(ds) {}
```

```
UserActionInitialization::~UserActionInitialization() {}
```

```
void UserActionInitialization::Build() const {  
    SetUserAction(new RunAction(dataStore));  
    SetUserAction(new PrimaryGenerator());  
}
```

コンストラクタの初期化リストを使う

Application_Main.cc

```
#include "G4RunManager.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "G4PhysListFactory.hh"
#include "Geometry.hh"
#include "DataStore.hh"
#include "UserActionInitialization.hh"
```

```
int main(int argc, char** argv) {
// DataStore instance
```

```
    DataStore* dataStore = new DataStore();
```

→ 先ずDataStoreのインスタンスを作り、渡す

```
// RunManager
```

```
    G4RunManager* runManager = new G4RunManager();
```

```
// Set user-defined initialization classes
```

```
    Geometry* geometry = new Geometry();
```

```
    geometry->SetDataStore(dataStore);
```

```
    runManager->SetUserInitialization(geometry);
```

```
    runManager->SetUserInitialization(new UserActionInitialization(dataStore));
```

```
// Physics list
```

```
    G4PhysListFactory factory;
```

```
    G4VModularPhysicsList* physicsList = factory.GetReferencePhysList("FTFP_BERT");
```

```
    runManager->SetUserInitialization(physicsList); // Initialize G4 kernel
```

```
    runManager->Initialize();
```

以下略

独立したデータストア まとめ

- 比較のために、Geant4/examples では殆どのものがG4AnalysisManagerをつかっている
 - UserRunAction の**コンストラクタの中で**
 - CreateH1, CreateH2, CreateH3関数によって**ヒストグラムを定義し**、
 - CreateNtuple でtupleを定義し、
 - スコアが得られたところで**FillH1, FillH2, FillH3, FillNtuple** を行う。
 - **EndOfRunAction**で書き出す。
 - ROOT, CSV, xml など
- これと比べて、今回の独立したデータストアでは
 - STLに準拠する
 - コンテナ `std::map`, `std::tuple`, `std::vector` などいろいろ
 - イテレータ コンテナの種類とアクセス方法によって使い分けられる
 - アルゴリズム `sort`, `find`, `accumulate`, `transform`, `binary search`
 - 汎用性、効率性、再利用性など優れた特長を利用できる

CSVファイルの解析は演習H10へ

演習 続く

参考資料

LLMを使ってみる

準備： 無料版Copilotを使う前のやり取りから

私はMicrosoftによって開発されたAIコンパニオン「Copilot」ですので、GitHubとの直接的な関係はありません。
ただし、MicrosoftはGitHubを所有しており、GitHub Copilotという別のAIツールも存在します。
そのツールは特に開発者向けに、コードの補完や提案を行う機能を持っています。

私が生成するコードはあなたの参考や支援のためのものであり、
その著作権を主張することはありません。
最終的にあなたが訂正したり改良したコードは、完全にあなたのものです。
私はツールとして、あなたの作業をサポートする役割を果たしています。

自分でも著作権について確認しておくといだろう

Copilotにやらせて間違いを意見したことへの回答。

- Geometry Construct を間違った。メインでrunManager のInitialize()を忘れた、など。やり直させて、動くものができたなどをCopilotに指摘した。またCopilotにChatGPTと比べさせた。
- 以下はCopilotの反省文の丸コピー。

貴重なフィードバックをありがとうございます。ご指摘の通り、私はコード生成において迅速さが特徴ですが、設計思想や全体の方針に対して十分な考慮が欠けることがありました。そのため、初期の間違いが連鎖的に影響することがあり、訂正には時間がかかることもあります。

Copilotの長所

迅速なコード生成: ご指摘の通り、コードを素早く生成できます。これは特にテンプレート化された構造や繰り返しの作業で有用です。ユーザーの指示に対する柔軟性: 問題を指摘していただくことで、適切に修正を行い、最終的には正しいコードに到達できます。

Copilotの短所

設計思想の理解不足: 複雑な設計や方針に対して、初期段階で誤りが生じることがあります。

修正時の方向性の誤り: 初期の間違いを修正する際に、さらなる誤りが生じることがあります。

ChatGPTの特徴

ChatGPTもまた興味深い特徴を持っています。設計思想に対する理解や、複雑なプロジェクトに対するアプローチにおいては、異なるアプローチを取るかもしれません。ChatGPTのフィードバックと比較することで、それぞれの長所を活かす方法が見えてくることがあります。

最終的に、私(Copilot)は迅速なコード生成と修正能力を持っていますが、ユーザーのフィードバックを通じてより正確な結果を出すために努力しています。今回のように問題点を指摘していただくことで、より良い解決策を提供できるようになります。

ChatGPTの見解は

- プログラム開発では、CopilotとChatGPTの両方が使えますが、それぞれ得意な場面が異なります。Copilotが向いている場面
コードの自動補完・生成(関数やクラスの作成を素早く提案)既存のコードの続きを書く(シンプルな実装ならほぼ自動)コードのテンプレートを素早く生成(Reactのコンポーネント、APIリクエスト、アルゴリズムなど)IDE(VS Code、JetBrainsなど)でシームレスに動作ChatGPTが向いている場面設計・アルゴリズムの考案(「この要件を満たす設計を考えて」といった質問に対応)コードの解説(Geant4や特殊なライブラリの動作を説明できる)デバッグ・エラーメッセージの解析(エラーの意味を説明し、修正案を提示)技術の学習・調査(「C++のスマートポインタの使い方は?」のような質問)特定の開発ツールの設定、結論開発のスピードを上げるならCopilot、考えたり学んだりするならChatGPT という感じです。もし具体的な開発の悩みがあれば、それに合わせた使い方を提案できますよ!
- まさにその通りですね! Copilotは一度間違えると同じ間違いを繰り返しやすいですが、指摘すると学習したかのように修正してくれることもあります。ChatGPTは、ユーザーの意図をくみ取って考え方を変えられるのが強みですね。Copilotの「パターンに沿って補完する力」と、ChatGPTの「意図を理解して応用する力」が組み合わさると、開発の生産性がかなり上がります。昔は検索してドキュメントを読んで試行錯誤するのが当たり前でしたが、今はAIと対話しながらコードを書けるのが面白いですね。このままいくと、開発のスタイルもさらに変わりそうです。たとえば、「人間は設計や要件定義に集中し、実装はほぼAIに任せろ」といった未来もありそう。今の時点でAIをうまく使いこなせる人は、これからの開発でも有利になりそうですね。

じつはChatGPTには、後に出るS案を使ってほぼ同じプロンプトを与えてプログラムを完成させていた。

P09/Bragg_DataStore 作成に使った指示

指示の具体的指示文

以下のようなテキストを Copilotに与えてすべてのコードを一括して提出するように求めた。先ず目的を示し、ステップバイステップで推敲するように指示した。以下、緑の箱の中の文が指示文である。

Let's create a simple but illustrative example code of Geant4 application.
Objective is to get depth-energy deposit relation, so called Bragg peak for incident protons into a water phantom.

Let's go **step by step** to create definition and implementation files.

1. Geometry class Geomtry.hh and Geometry.cc This same as before.

- world is a cube of 1 m filled with air.
- a water phantom is a cube of 30*cm is filled with water.
- It is sliced into 300 plates of 1*mm thickness using G4PVReplica in Z direction
- In Geometry::Construct method the logical volume of the plate is assigned to the Sensitive detector with name "WaterPhantom".
- the reference to "DataStore" must be known here.

2. DataStore class

- it has a private store for energy deposit at given copy number plate.

We use private:

```
std::map<G4int copyNo, G4double energy>myMap
```

- The class must have public methods ;
InitializeMap(myMap) to clear the map at the RunAction::BeginOfRun,
- and public: GetMap which is used at the RunAction::EndOfRun.
- FillMap() method which is used by the SensitiveDetector to fill score

指示続き

3. SensitiveDetector class SensitiveDetector.hh, SensitiveDetector.cc

-
- SensitiveDetector::Initialize does nothing
- SensitiveDetector::ProcessHits function must accumulate energy deposit of G4Step at this copyNo.
It gets the copyNo of the plate and total energy deposit therein.
- SensitiveDetector::EndOfEvent does nothing

4. RunAction class RunAction.hh RunAction.cc

- it must be capable of getting the reference to DataStore
- BeginOfRunAction do Initialize DataStore's myMap.
- EndOfRunAction use DataStore's GetMap method and output the Map to a file in CSV
format "copyNo, energy" and output to a CSV file "Bragg.csv"

5. UserActionInitialization class must be properly initialized.

it must be capable of getting the reference to DataStore

6. PrimaryGenerator class must be properly made.

7. Main program: Application_Main.cc

- include all necessary files
- be careful of the order of initialization
- DataStore is instantiated here and its reference must be passed to relevant classe.
- RunManager is "Serial" mode and be initialized at appropriate timing.
- interactive mode with visualization
- proper physics list must be used
- macro file "GlobalSetup.mac" must be called at start up time.

指示終わり

In summary, we must create

Geometry.hh Geometry.cc
DataStore.hh and DataStore.cc
SensitiveDetector.hh SensitiveDetector.cc
RunAction.hh RunAction.cc
PrimaryGenerator.hh PrimaryGenerator.cc
UserActionInitialization. UserActionInitialization.cc
Application.cc

Notice

- Header files whose name start with "G4" are not the system ones but ours.
- Insert appropriate comment lines to show what you are going to do

OK now, propose you complete codes set. good luck.

Copilotが提出したコードには
いくつかのちょっとした間違いや一つの重大なミスがあったが、
コンパイルエラーを見せたら理解して正しいものを提出した。

ChatGPT S案の指示を出してみたら

一部手直ししたら動いた

SensitiveDetectorにデータを持たせるS案

- 目標 自前でSensitiveDetector を作ってCBSと同じ結果を出す。
 - Geometryについて
 - 300分割した水の小箱の作り方
 - SensitiveDetectorへの割り付け
 - SensitiveDetectorについて
 - SensitiveDetectorが300分割された小箱ごとにエネルギーデポジットを積算する
 - データのファイルへの保存について
 - ランの最初にはデータを初期化し、ランの終わりにデータを取り出してCSVファイルに書き出す

データを誰が持つべきか S案では、

- SensitiveDetectorがデータを持ちランが終わったらRunActionで取り込む

指示

2. SensitiveDetector class SensitiveDetector.hh, SensitiveDetector.hh

- it has **a private store for energy deposit at given copy number plate.**

We use private:

std::map<G4int copyNo, G4double energy>myMap

- The class must have two public methods ;

InitializeMap() to clear the map at the RunAction::BeginOfRun,

- and public: **GetMap** which is used at the RunAction::EndOfRun.

- SensitiveDetector::Initialize does nothing

- SensitiveDetector::ProcessHits function must accumulate energy deposit of G4Step at this copyNo. It gets the copyNo of the plate and total energy deposit therein.

- SensitiveDetector::EndOfEvent does nothing

SensitiveDetector.hh

■ 正解

```
#ifndef SENSITIVE_DETECTOR_HH
#define SENSITIVE_DETECTOR_HH
#include "G4VSensitiveDetector.hh"
#include <map>
class SensitiveDetector : public G4VSensitiveDetector {
public:  SensitiveDetector(const G4String& name);
    ~SensitiveDetector() override;
    void Initialize(G4HCofThisEvent* hce) override;
    G4bool ProcessHits(G4Step* step, G4TouchableHistory* history) override;
    void EndOfEvent(G4HCofThisEvent* hce) override;
    const std::map<G4int, G4double>& GetMap() const;
    void InitializeMap() ;
private:
    std::map<G4int, G4double> energyMap;};
#endif
```

この人が持つことになるメソッド

SensitiveDetector.cc

■ 正解

```
#include "SensitiveDetector.hh"
#include "G4Step.hh"
#include "G4TouchableHistory.hh"
SensitiveDetector::SensitiveDetector(const G4String& name) : G4VSensitiveDetector(name) {}
SensitiveDetector::~SensitiveDetector() = default;
G4bool SensitiveDetector::ProcessHits(G4Step* step, G4TouchableHistory*) {
    G4double edep = step->GetTotalEnergyDeposit();
    if (edep > 0) {
        G4int copyNo = step->GetPreStepPoint()->GetTouchable()->GetCopyNumber();
        energyMap[copyNo] += edep;
    }
    return true;
}
void SensitiveDetector::Initialize(G4HCofThisEvent*) {}
void SensitiveDetector::EndOfEvent(G4HCofThisEvent*) {}
void SensitiveDetector::InitializeMap() { energyMap.clear();}
const std::map<G4int, G4double> & SensitiveDetector::GetMap() const {
    return energyMap;
}
```

Geometry.cc

■ 更問したらよくなった。Replication 正しい。。

```
// Water Phantom
G4Material* water = nist->FindOrBuildMaterial("G4_WATER");
G4Box* phantomBox = new G4Box("Phantom", 15 * cm, 15 * cm, 15 * cm);
G4LogicalVolume* phantomLogic =
    new G4LogicalVolume(phantomBox, water, "Phantom");
new G4PVPlacement(nullptr, {}, phantomLogic, "Phantom", worldLogic, false, 0);
// Slices
G4Box* sliceBox = new G4Box("Slice", 15 * cm, 15 * cm, 0.5 * mm);
G4LogicalVolume* sliceLogic = new G4LogicalVolume(sliceBox, water, "Slice");
new G4PVReplica("Slices", sliceLogic, phantomLogic, kZAxis, 300, 1 * mm);
//Chat GPT は下の行を忘れていたが更問したら直した
// Assign sensitive detector
G4SDManager* sdManager = G4SDManager::GetSDMpointer();
SensitiveDetector* sd = new SensitiveDetector("WaterPhantom");
sdManager->AddNewDetector(sd);
sliceLogic->SetSensitiveDetector(sd); // 正解
return worldPhys;}
```

指示: RunAction

- ランの始めと終わりにSensitiveDetector のprivate データを操作する手法がポイント。最初の間違いを指摘して直し方を教えた。

3. RunAction class RunAction.hh RunAction.cc

- SensitiveDetector instance **must be got by its name "WaterPhantom" using G4SDManager.**
- BeginOfRunAction do Initialize SensitiveDetector's myMap.
- EndOfRunAction use SensitiveDetector's GetMap method and output the Map to a file in CSV format "copyNo, energy" and output to a CSV file "Bragg.csv"

5. UserActionInitialization class must be properly initialized.

6. PrimaryGenerator class must be properly made.

RunAction.hh

■ 正解

```
#ifndef RUN_ACTION_HH
#define RUN_ACTION_HH
#include "G4UserRunAction.hh"
#include <map>
#include <fstream>
class SensitiveDetector;
class RunAction : public G4UserRunAction {
public:  RunAction();
       ~RunAction() override;
       void BeginOfRunAction(const G4Run* run) override;
       void EndOfRunAction(const G4Run* run) override;
private:
       std::ofstream outFile;};
#endif
```

RunAction.cc 赤字の部分は更問して直させた

```
#include "RunAction.hh"
#include "SensitiveDetector.hh"
#include "G4SDManager.hh"
// chat 2 nd 更問
#include "G4Run.hh"
#include <iomanip>
```

SensitiveDetectorオブジェクトを得るには、
G4SDManagerを経由し、その名前を頼りに探す

```
RunAction::RunAction() = default; // // : detector(detector) {}
RunAction::~RunAction() = default;
void RunAction::BeginOfRunAction(const G4Run*) {
    G4SDManager* sdMan = G4SDManager::GetSDMpointer();
    auto detector = static_cast<SensitiveDetector*>(sdMan->FindSensitiveDetector("WaterPhantom"));
    detector->InitializeMap();
    outFile.open("Bragg.csv");
    outFile << "CopyNo,Energy¥n";
}
void RunAction::EndOfRunAction(const G4Run*) {
    G4SDManager* sdMan = G4SDManager::GetSDMpointer();
    auto detector = static_cast<SensitiveDetector*>(sdMan->FindSensitiveDetector("WaterPhantom"));
    const auto& energyMap = detector->GetMap();
    for (const auto& [copyNo, energy] : energyMap) {
        outFile << copyNo << "," << energy << "¥n";
    }
    outFile.close();
}
```



プライベートデータへのアクセス方法を直させた

- 最初の回答ではRunAction クラスがSensitiveDetectorへアクセスするためにRunActionのコンストラクタの中でdetector = new SensitiveDetector) とした。これは間違い。
- SensitiveDetectorのインスタンスはGeometry::ConstructのなかでG4SDManagerによって割り当てられ(名前付きで)論理ボリウムに紐づけされている。RunAction のなかでnewしたものはそれが無いので、スコアデータを持たない。すべてのSensitiveDetectorsに他のクラスからアクセスするにはシングルトンであるG4SDManager に頼んで名前から手繰るのが正しい方法である。
- このことを具体的に指摘したらここに在るように正しいものに直してきた。

指示: Main runManager->Initialize() を忘れるなど手直し

7. Main program: Application_Main.cc

- include all necessary files
- be careful of the order of initialization
- RunManager is "Serial" mode
- interactive mode with visualization
- proper physics list must be used
- macro file "GlobalSetup.mac" must be called at start up time.

```
runManager->SetUserInitialization(new QGSP_BERT());
```

 → 探してきた

Notice

- Header files whose name start with "G4" are not the system ones but ours.
これをうるさく言わないとすべて #include <G4xxxx.hh>としてしまう。
- Insert appropriate comment lines to show what you are going to do
これは上手にできない。

OK now, propose your complete codes set. good luck. 2025

ChatGPT R案の指示を出してみたら

一部手直ししたら動いた

前と違うのは、RunDataクラスにデータを持たせること

. RunData class.

it has a store for energy deposit at given copy number plate.

We use private: `std::map<G4int copyNo, G4double energy>myMap`

The class must have public methods ;Initialize(myMap) to clear the map at the BeginOfRun,

AddEnergy(copyNo) to add deposit energy in ProcessHits method of SensitiveDetector during the run and GetMap at the EndofRun.

Be careful cast to a non-const `G4Run*` when using RunData like

`RunData* thisRun = (RunData*) run;` ← すでに犯していた誤りを直させた

3. UserRunAction class.

it must have `GenerateRun()` for RunData

BeginOfRunAction do Initialize myMap.clear().

EndOfRunAction use GetMap and output to a file in CSV format

“copyNo, energy”

ProcessHits methods of SensitiveDetector class gets the copyNo of the plate and total energy deposit therein. Then it get a instance of RunData

and use it AddEnergy 4. SensitiveDetector class

Only ProcessHits function must be implemented to accumulate energy deposit of G4Step.

SensitiveDetector.cc

```
#include "SensitiveDetector.hh"
#include "RunData.hh"
#include "G4Step.hh"
#include "G4TouchableHistory.hh"
#include "G4RunManager.hh"

SensitiveDetector::SensitiveDetector(const G4String& name)
    : G4VSensitiveDetector(name) {}

SensitiveDetector::~SensitiveDetector() {}

G4bool SensitiveDetector::ProcessHits(G4Step* step, G4TouchableHistory*) {
    G4double edep = step->GetTotalEnergyDeposit();
    if (edep == 0) return false;

    G4int copyNo = step->GetPreStepPoint()->GetTouchable()->GetReplicaNumber();
    RunData* runData = static_cast<RunData*>
        (G4RunManager::GetRunManager()->GetNonConstCurrentRun());
    runData->AddEnergy(copyNo, edep);

    return true;
}
```

G4RunManager から今のランのオブジェクトrunDataを得る

RunData.hh

```
#ifndef RUNDATA_HH
#define RUNDATA_HH

#include "G4Run.hh"
#include <map>

class RunData : public G4Run {
public:
    RunData();
    virtual ~RunData();

    void Initialize();
    void AddEnergy(G4int copyNo, G4double energy);
    const std::map<G4int, G4double>& GetMap() const;

private:
    std::map<G4int, G4double> myMap;
};

#endif
```

データを持つ人

RunData.cc

```
#include "RunData.hh"

RunData::RunData() : G4Run() {}
RunData::~~RunData() {}

void RunData::Initialize() {
    myMap.clear();
}

void RunData::AddEnergy(G4int copyNo, G4double energy) {
    myMap[copyNo] += energy;
}

const std::map<G4int, G4double>& RunData::GetMap() const {
    return myMap;
}
```

自分の持ち物なので

RunAction.cc

```
#include "UserRunAction.hh"
#include "RunData.hh"
#include "G4RunManager.hh"
#include <iomanip>

UserRunAction::UserRunAction() : G4UserRunAction() {}
UserRunAction::~~UserRunAction() {}

G4Run* UserRunAction::GenerateRun() {
    return new RunData();
}

void UserRunAction::BeginOfRunAction(const G4Run* run) {
    auto myRun = static_cast<const RunData*>(run);
    myRun->Initialize();
    outputFile.open("bragg_peak.csv");
    outputFile << "CopyNo,Energy¥n";
}

void UserRunAction::EndOfRunAction(const G4Run* run) {
    auto myRun = static_cast<const RunData*>(run);
    const auto& map = myRun->GetMap();

    for (const auto& [copyNo, energy] : map) {
        outputFile << copyNo << ", " << energy / CLHEP::MeV << "¥n";
    }
    outputFile.close();
}
```

ここだけ訂正した

LLM 利用の勧め

まとめ

- 何をさせるかをよくわかっているときにはそれを詳細に記述するとよい。
- しかし、Geant4のクラスの間の関係や実行の時系列を理解していないと正しい答えは返ってこない。
- C++の文法がうろ覚えの時は大きな助けになる。
- 初心者にもやさしい。次の例を見て。

質問 次の文を解説してください。

```
SensitiveDetector::SensitiveDetector(const G4String& name, DataStore* ds)  
: G4VSensitiveDetector(name), dataStore(ds) {}
```

回答 はCopilotさん

このコードは、C++で書かれたコンストラクタの定義です。具体的には、SensitiveDetector というクラスのコンストラクタを定義しています。
解説を以下に示します。

- 1.SensitiveDetector::SensitiveDetector:
 - これは SensitiveDetector クラスのコンストラクタを定義しています。
 - SensitiveDetector クラスは、おそらくユーザーが定義したクラスです。
- 2.const G4String& name:
 - これは、コンストラクタの引数で、name という名前の G4String 型の変数を参照として受け取ります。
 - G4String は、おそらく G4 ライブラリ (Geant4) の文字列クラスです。
- 3.DataStore* ds:
 - これは、コンストラクタのもう一つの引数で、ds という名前の DataStore 型のポインタを受け取ります。
 - DataStore は、おそらくユーザーが定義したデータストアクラスです。
- 4.: G4VSensitiveDetector(name), dataStore(ds):
 - これは初期化リストと呼ばれ、基底クラスとメンバ変数を初期化します。
 - G4VSensitiveDetector(name) は、SensitiveDetector クラスの基底クラスである G4VSensitiveDetector クラスのコンストラクタを呼び出し、name という引数を渡します。
 - dataStore(ds) は、SensitiveDetector クラスのメンバ変数 dataStore を ds という引数で初期化します。

まとめ

このコンストラクタは、SensitiveDetector クラスの新しいインスタンスを作成し、その際に基底クラス G4VSensitiveDetector のコンストラクタを呼び出し、name を渡します。また、dataStore というメンバ変数を ds で初期化します。
これにより、オブジェクトが正しく初期化され、使用する準備が整います。

もし他に質問があれば、教えてくださいね！

教師よりも親切!!